# ClaimPilot AI

# Complete Technical Documentation

Author: ClaimPilot AI Engineering Team

Date: June 30, 2026

Version: 0.2.0 (API) / 0.1.0 (frontend package)

Synthetic demo system — not for production PHI. All claims traceable to codebase as of June 2026.

# Table of Contents

# ClaimPilot AI — Complete Technical Documentation

**Author:** ClaimPilot AI Engineering Team

**Date:** June 30, 2026

**Version:** Backend API 0.2.0 · Frontend package 0.1.0

## 1. Executive Summary

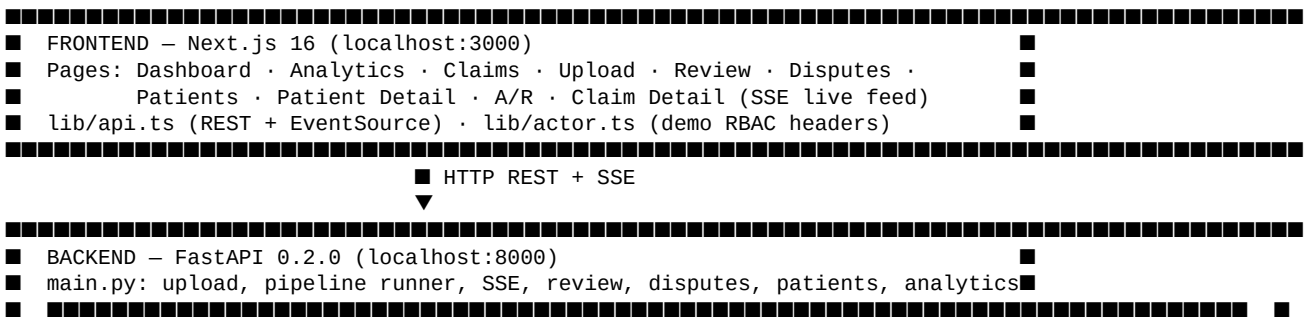
ClaimPilot AI is a multi-agent healthcare claims automation platform built as an intern engineering demo. It simulates the revenue-cycle workflow of a medical billing office: a superbill (PDF or image) is uploaded, structured claim data is extracted, coverage is verified, coding is validated, payer edits are applied, denial risk is scored, the claim is submitted to a simulated payer, remittance is reconciled, and patient balances are collected. Seven specialized pipeline agents share a single typed **ClaimState** (Pydantic v2) object, orchestrated by a **LangGraph** supervisor with conditional routing and human-in-the-loop (HITL) interrupts.

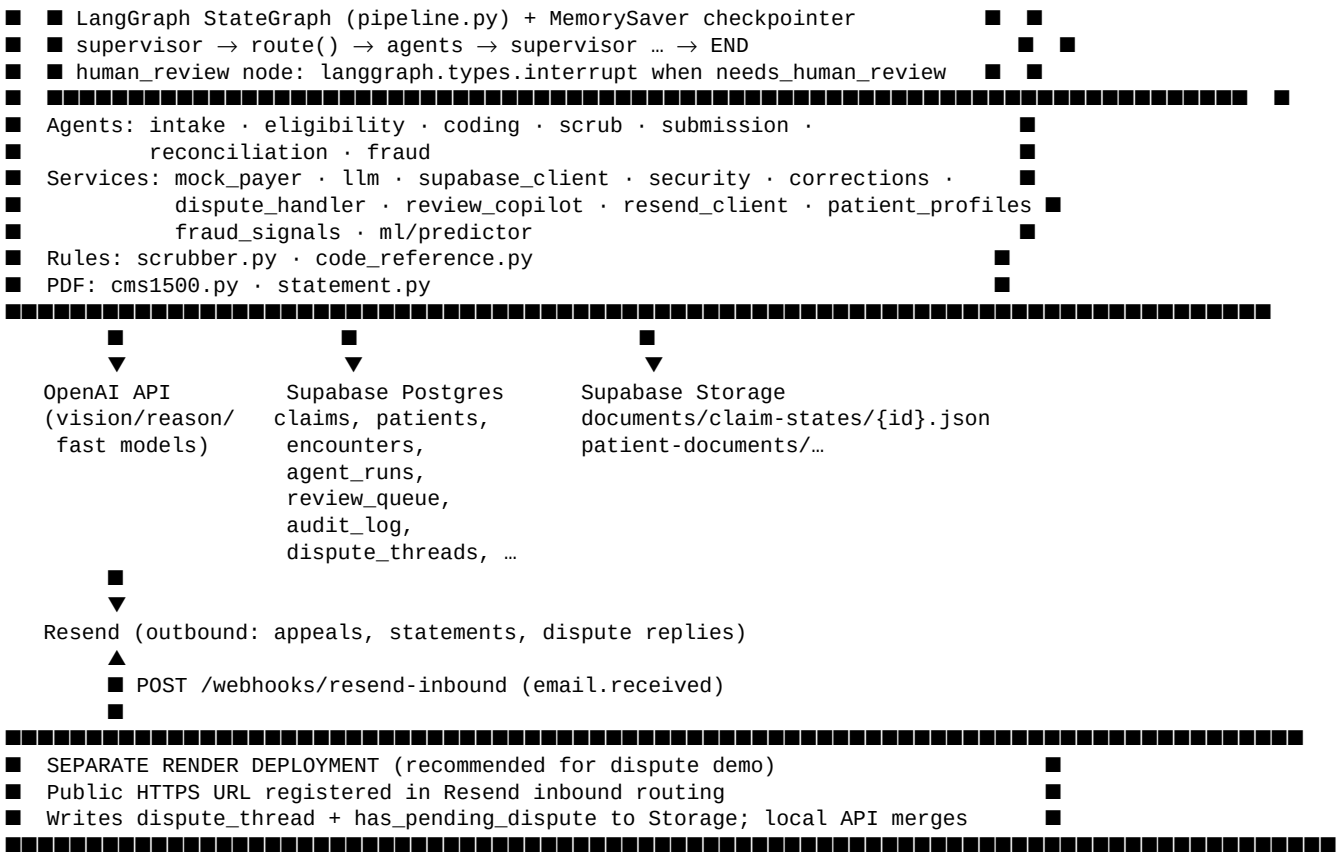
The system is designed for billing operations staff, practice managers, and engineering reviewers evaluating agentic RCM (revenue cycle management) patterns. It solves the problem of fragmented, manual claim handling by chaining deterministic billing rules, mock payer adjudication, machine-learning risk scoring, GPT vision/reasoning for unstructured documents and correspondence, and a review queue with role-based approval gates — all observable in real time via server-sent events (SSE).

Architecture at a glance: a **Next.js** frontend (App Router, TypeScript, Tailwind, shadcn/ui) talks to a **FastAPI** backend on port 8000. **Supabase** provides Postgres (claims, patients, audit, review queue) and Storage (full ClaimState JSON snapshots). **OpenAI** models (configured via `MODEL_VISION`, `MODEL_REASONING`, `MODEL_FAST` env vars — never hardcoded) power extraction, coding review, appeals, dispute replies, and the Review Copilot. **Resend** sends appeal, patient statement, and dispute emails; inbound dispute webhooks are intended to run on a **separate Render deployment** with public HTTPS while the local backend reads dispute state from Storage. All patient and claim data is **synthetic**; the system is not certified for production use with real PHI.

## 2. System Architecture

### 2.1 Component diagram






### 2.2 Data flow: upload to reconciliation

1. **Upload** — `POST /claims/upload` or `/claims/upload-batch` saves file to `data/synthetic/uploads/{claim_id}{suffix}`, inserts a `claims` row (`status: draft`), initializes `ClaimState`, starts `_run_pipeline` as asyncio background task.
2. **Intake** — If `status=draft` and document present but no lines: vision model extracts superbill fields into `ClaimState`; low confidence → `needs_review`.
3. **Eligibility** — Mock 270/271 via `check_eligibility`; terminated coverage → review on first pass.
4. **Coding** — GPT structured `CodingReview` validates ICD-10/CPT linkage; sets `status=coded`.
5. **Scrub** — Rules engine + ML denial risk + CMS-1500 PDF; hard errors or high risk → review queue.
6. **Human review (optional)** — LangGraph `interrupt`; reviewer calls `POST /claims/{id}/resume`.
7. **Submission** — `adjudicate_claim` (rule-based mock payer, **not ML**); clinical denials → appeal letter; administrative → review.
8. **Reconciliation** — `generate_era` (835/ERA simulation); variance over tolerance → review; else `reconciled`, patient A/R, statement PDF, optional email.
9. **Fraud** — Isolation Forest + cross-claim signals; informational on terminal states.
10. **Persistence** — After each graph node: SSE fan-out, `save_claim_state` to Storage, `agent_runs` rows, flat `claims` columns, `upsert_patient_from_claim`.

## 3. Technology-to-Feature Map

| Feature | Technology | File(s) |
|---|---|---|
| Superbill data extraction | OpenAI vision (`MODEL_VISION`), Pydantic `SuperbillExtraction`, `structured_call`, pdf2image + Poppler for PDF→PNG | `agents/intake.py`, `schemas/superbill.py`, `services/llm.py` |
| Low-confidence HITL gate | `CONFIDENCE_THRESHOLD` (default 0.85) | `agents/intake.py` |
| Eligibility 270/271 simulation | Deterministic `check_eligibility`, MD5-seeded member outcomes | `agents/eligibility.py`, `services/mock_payer.py` |
| ICD-10/CPT coding review | OpenAI `MODEL_REASONING`, `CodingReview` schema | `agents/coding.py` |
| Pre-submission scrubber | Python rules: NPI Luhn, NCCI, MUE, modifiers, TFL, prior auth, LCD | `rules/scrubber.py`, `rules/code_reference.py` |
| Denial risk prediction | scikit-learn `GradientBoostingClassifier`, SHAP `TreeExplainer`, features from pre-submission observables | `ml/train.py`, `ml/features.py`, `ml/predictor.py`, `agents/scrub.py` |
| CMS-1500 PDF | ReportLab canvas | `pdf/cms1500.py`, `agents/scrub.py` |
| Mock payer adjudication | **Rule-based** deterministic engine (NOT ML) | `services/mock_payer.py`, `agents/submission.py` |
| Appeal letter drafting | OpenAI `MODEL_REASONING`, `text_call`, plain-text strip | `agents/submission.py`, `services/llm.py` |
| 835/ERA reconciliation | `generate_era`, variance vs `RECON_VARIANCE_TOLERANCE` | `agents/reconciliation.py`, `services/mock_payer.py` |
| Patient statement PDF + email | ReportLab `generate_statement`, Resend attachment | `pdf/statement.py`, `services/resend_client.py` |
| Fraud / anomaly | Isolation Forest + in-process cross-claim statistics | `agents/fraud.py`, `ml/predictor.py`, `services/fraud_signals.py` |
| LangGraph supervisor | `StateGraph(ClaimState)`, `MemorySaver`, `route()`, `interrupt` | `graph/pipeline.py` |
| SSE real-time events | `sse-starlette` `EventSourceResponse`, per-claim subscriber queues | `main.py` |
| Human review resume | `POST /claims/{id}/resume`, checkpointer new thread id | `main.py`, `graph/pipeline.py` |
| RBAC demo | Header-based `Actor`, `can_approve` thresholds | `services/security.py` |
| Audit trail | `audit_log` table, append-only triggers (migration 0002) | `services/supabase_client.py`, `main.py` |

| Feature | Technology | File(s) |
|---|---|---|
| Corrected claims (freq 7/8) | `build_corrected_claim`, re-pipeline from `coded` | `services/corrections.py`, `main.py` |
| Review Copilot | Grounded markdown context, `MODEL_FAST` / `MODEL_REASONING`, `CopilotResponse` | `services/review_copilot.py` |
| Dispute email thread | Resend inbound webhook, Receiving API, GPT reply, escalation keywords | `main.py`, `services/dispute_handler.py`, `services/resend_client.py` |
| Patient profiles | Supabase CRUD, auto `upsert_patient_from_claim` | `services/patient_profiles.py`, migration 0007 |
| Analytics | SQL aggregates over `claims`, `review_queue`, `agent_runs` | `main.py` `/analytics` |
| Batch upload | Parallel `asyncio.create_task` per file | `main.py` |
| ClaimState durability | Supabase Storage JSON (`documents/claim-states/`) | `services/supabase_client.py` |

## 4. The 7-Agent Pipeline

### 4.1 Intake Agent

**Purpose:** Extract patient, payer, provider, date of service, and service lines (CPT, modifiers, ICD-10, units, charges) from an uploaded superbill PDF or image.

**Technology:** OpenAI vision model from `MODEL_VISION` (default `gpt-4o-mini` per `.env.example`); `client.beta.chat.completions.parse` with `SuperbillExtraction` schema; PDFs converted via `pdf2image.convert_from_path` (Poppler, `POPPLER_PATH` or default `C:\poppler\...`).

**ClaimState reads/writes:** Reads `document_storage_path`. Writes `patient_*`, `provider_*`, `date_of_service`, `claim_lines`, `total_charge`, `extraction_confidence`, `low_confidence_fields`, `status` (`extracted` or `needs_review`), `agent_events`.

**Human review trigger:** Any header field confidence below `CONFIDENCE_THRESHOLD` (default 0.85). `review_reason`: `"Low confidence on: {fields}"`.

**Example output summary:** `"Extracted superbill for Smith, James: 3 line(s), total $474.00. All fields high-confidence."` or `"Extraction complete — 2 low-confidence field(s) flagged: patient_dob, provider_npi."`

### 4.2 Eligibility Agent

**Purpose:** Simulate X12 270/271 eligibility — active coverage, plan name, copay, coinsurance, deductible, prior-auth CPT list, auth-on-file flag.

**Technology:** Deterministic `check_eligibility(payer_name, member_id)` in `mock_payer.py` — ~4% terminated coverage, tiered plans (PPO Gold/Silver, HMO Value), auth-on-file ~85% when auth required.

**ClaimState reads/writes:** Writes `eligibility_checked`, `eligibility_active`, `plan_name`, `copay`, `coinsurance`, `deductible_*`, `prior_auth_cpts`, `prior_auth_on_file`. Advances to `extracted` when active.

**Human review trigger:** Inactive coverage on first check: `"Coverage terminated — member not eligible on date of service"`. After reviewer approval, proceeds with note that CARC 27 is expected.

**Example output:** `"271 response: active coverage, plan Aetna PPO PPO Silver. Copay $45.00, coinsurance 20%, deductible remaining $1,200.00 of $1,500.00."`

### 4.3 Coding Agent

**Purpose:** Validate ICD-10-CM format, CPT format, medical-necessity linkage, modifier appropriateness, unbundling.

**Technology:** OpenAI `MODEL_REASONING` with structured `CodingReview` (validated, issues, suggestions, summary).

**ClaimState reads/writes:** Writes `coding_issues`, `coding_validated`, `status=coded` (unless already in review). Does not alone trigger HITL.

**Human review trigger:** None directly.

**Example output:** `"Coding validated — all 3 lines pass ICD-10/CPT checks."` or `"Coding review found 1 issue(s): ICD-10 E11.9 may not support CPT 93000 without cardiac indication."`

### 4.4 Scrub Agent

**Purpose:** Run full pre-submission scrubber, score ML denial risk, generate CMS-1500 PDF.

**Technology:** `scrub_claim()` rules engine; `predict_denial_risk()` (GradientBoosting + SHAP); ReportLab `generate_cms1500` in thread pool.

**ClaimState reads/writes:** `scrub_findings`, `scrub_passed`, `denial_risk`, `denial_risk_factors`, `cms1500_path`, `status` (`scrubbed` or `needs_review`).

**Human review triggers:**

- Scrub **errors** (severity `error`): `"Scrubber blocked submission: N hard error(s) — [RULE] message"`

- `denial_risk >= DENIAL_RISK_THRESHOLD` (default 0.60): `"Denial risk XX% exceeds threshold"`
- Inserts row into `review_queue` when triggered.

**Example output:** `"Clean scrub — all 3 line(s) pass NCCI, MUE, modifier, identifier, and coverage edits. CMS-1500 generated. ML denial risk: 32%."`

### 4.5 Submission Agent

**Purpose:** Submit to mock clearinghouse (837P analogue), adjudicate, draft appeals or route administrative denials to review.

**Technology: Rule-based** `adjudicate_claim()` — NOT machine learning. GPT `MODEL_REASONING` + `text_call` for appeal letters on appealable CARCs: `{50, 97, 151, 197, 11, 4, 96}`.

**ClaimState reads/writes:** `clearinghouse_ref`, `adjudication`, `amount_expected`, `carc_code`, `rarc_code`, `denial_reason`, `appeal_letter`, `status` (`submitted`, `denied`, `appealed`).

**Human review trigger:** Non-appealable administrative CARCs (16, 18, 27, 29, 22, 109) → review with corrective action in `recon_notes`.

**Example output (paid):** `"Claim accepted by clearinghouse. Ref: CLH-A1B2C3D4. All 3 line(s) payable — expected payment $142.50 after contractual adjustment and patient responsibility. Awaiting ERA."`

**Example output (appeal):** `"Appeal letter drafted for CARC 50 denial — 287 words, citing medical necessity for E11.9, I10. Appeal letter ready for review and sending."`

### 4.6 Reconciliation Agent

**Purpose:** Parse simulated 835/ERA, compare expected vs paid, open patient A/R, generate statement, optionally email patient.

**Technology:** `generate_era()` — deterministic; ~10% underpayment injection; `finalize_patient_ar`, ReportLab statement, Resend `send_patient_statement_email`.

**ClaimState reads/writes:** `era`, `amount_paid`, `patient_responsibility`, `recon_variance`, `recon_discrepancy`, `patient_balance`, `ar_status`, `patient_statement_path`, `statement_date`, `status` (`reconciled` or `needs_review`).

**Human review trigger:** `variance_pct > RECON_VARIANCE_TOLERANCE` (default 5%): `"Payment variance $X.XX (Y%) exceeds tolerance"`.

**Example output:** `"Reconciliation complete. $142.50 posted from check CHK123456 — matches contracted rate ($474.00 billed, CO-45 contractual adjustment applied). Patient statement generated — balance due $45.00 (copay/deductible/coinsurance) posted to patient A/R."`

### 4.7 Fraud Agent

**Purpose:** Post-adjudication billing anomaly scan — single-claim shape + cross-provider patterns.

**Technology:** Isolation Forest (`anomaly_model.pkl`) on charge/lines/dx/MUE; `fraud_signals.evaluate()` for charge z-score, duplicate clone, E/M upcoding skew (≥80% level 4/5), volume spike (≥15 claims/DOS/NPI). Final score = `max(model, cross)`.

**ClaimState reads/writes:** `anomaly_score`, `anomaly_reasons`. Does not block pipeline or trigger review.

**Human review trigger:** None (informational only).

**Example output:** `"No anomalies detected. Billing pattern within normal parameters (model 12%, cross-claim 0%)."`

## 5. Machine Learning Systems

### 5.1 Denial risk model

**Training (`ml/train.py`):**

- Generates **8,000** synthetic claims via `_sample_claim()` with realistic error mix (unsupported dx, missing mod 25, MUE violations, invalid NPI, etc.).
- Labels from **actual** `adjudicate_claim()` outcomes (claim-level or any line denied) — not a hand-written formula.
- Model: `GradientBoostingClassifier(n_estimators=300, max_depth=4, learning_rate=0.05)`.
- Metrics printed and stored: accuracy, precision, recall, AUC; training **asserts AUC > 0.75**.
- Saved to `backend/app/ml/denial_model.pkl` with `feature_columns` and `metrics`.

**Features (`ml/features.py`):** Pre-submission observables only — `charge_amount`, `num_lines`, `num_dx_codes`, `dos_age_days`, `npi_valid`, `member_id_present`, `em_mod25_missing`, `bundled_99000`, `unsupported_dx_lines`, `units_over_mue`, `auth_required_cpt_present`, `near_filing_limit`, payer one-hots, CPT presence flags. Deliberately excludes hidden payer state (auth on file, termination).

**Inference (`ml/predictor.py`):** `predict_proba` → risk score; SHAP top-3 factors translated to plain English with approximate `% risk` delta.

**Fallback:** If `denial_model.pkl` missing, returns `(0.5, ["Model not loaded"])`.

### 5.2 Anomaly / fraud detection

**Isolation Forest:** Trained on `[charge_amount, num_lines, num_dx_codes, units_over_mue]` with `contamination=0.08`. Score mapped via sigmoid of `decision_function`.

**Cross-claim signals (`fraud_signals.py`):** In-process rolling history (max 5000 claims); charge outlier (z ≥ 2.5 vs peers, min 8 samples); duplicate clone (same NPI + member + CPT set); E/M upcoding (≥80% high-level E/M over ≥4 claims); volume spike (≥15 claims same DOS/NPI).

### 5.3 Mock payer vs ML (critical distinction)

**The mock payer adjudication engine is entirely rule-based and deterministic** (`adjudicate_claim`, `_claim_level_denial`, `_line_denial`). It uses the same NCCI, MUE, LCD, and modifier rules as the scrubber. Same claim content always produces the same adjudication (MD5-seeded only for eligibility tiers, underpayments, check numbers).

**The denial-risk ML model is a separate pre-submission predictor** trained to approximate those rule outcomes from observable features. It does **not** adjudicate claims. Confusing the two is a common demo misread: submission/payment outcomes come from rules; denial risk is an ML estimate for routing to human review.

## 6. Rules Engine and Compliance Logic

All rules live in `rules/scrubber.py` (pre-submission) and mirror `services/mock_payer.py` (adjudication).

| Rule ID | Severity | Description | CMS / code basis |
|---|---|---|---|
| NPI-01 | error | Missing NPI (box 33a) | CARC 16 / RARC N290 |
| NPI-02 | error | NPI not 10 digits | CARC 16 |
| NPI-03 | error | NPI fails Luhn (80840 prefix) | CARC 16 / RARC N290 |
| SUB-01 | error | Missing member ID (box 1a) | CARC 16 / MA61 |
| SUB-02 | error | Missing patient name | Unprocessable |
| SUB-03 | warning | Missing DOB | Box 3 |
| SUB-04 | error | Missing payer | Routing failure |
| DOS-01 | error | Unparseable DOS | Box 24A |
| DOS-02 | error | Future DOS | Invalid |
| DOS-03 | error | DOB after DOS | Invalid |
| TFL-01 | error | Past payer filing limit | **CARC 29** (90–365 days by payer) |
| TFL-02 | warning | Within 14 days of limit | Timely filing risk |
| LN-01–04 | error | No lines, no dx pointer, invalid charge/units | Box 24 |
| CPT-01/02 | error/warn | Format / not in reference | Invalid code |
| ICD-01/02 | error/warn | Format / not in reference | Invalid dx |

| Rule ID | Severity | Description | CMS / code basis |
|---|---|---|---|
| MOD-01–03 | warn/error | Unknown mod, mod 25 on non-E/M, mod 59 on E/M | **CARC 4**, **CARC 97** |
| NCCI-01 | error | 99000 status-B bundled with E/M; 80048 bundled in 80053 | **CARC 97**, RARC N19/M15 |
| MOD-25 | error | E/M same day as 90471 without mod 25 | **CARC 97** |
| MUE-01 | error | Units > MUE max | **CARC 151**, RARC N362 |
| AUTH-01 | error | Auth-required CPT without auth on file | **CARC 197**, RARC N130 |
| LCD-01 | error | Diagnosis prefix does not support CPT | **CARC 50**, RARC N115 |

**Payer filing limits:** BlueCross 365d, Aetna 120d, United/Cigna 90d, Humana 180d, default 180d.

**Prior auth CPTs by payer:** Aetna/United/Humana require auth for 93000 (and Humana for 80053) per `PAYER_RULES`.

## 7. Human-in-the-Loop and Governance

### 7.1 Review queue

- Table: `review_queue` (`status`: open → approved/rejected).
- Populated by scrub (denial risk / errors), submission (administrative denial), reconciliation (variance), and intake/eligibility gates.
- API: `GET /review` merges DB rows with in-memory state for live fields.

### 7.2 RBAC (demo tier)

| Role | Approve authority |
|---|---|
| **biller** | Routine reviews only: charge ≤ `$500` (`BILLER_APPROVAL_MAX_CHARGE`), denial risk < 75% (`BILLER_APPROVAL_MAX_RISK`), not payment-variance write-offs |
| **supervisor** | All approvals |
| **manager** | All approvals (default if headers missing) |

Demo users (`GET /auth/users`): Jordan Lee (biller), Sam Rivera (supervisor), Alex Morgan (manager). Frontend sends `X-Actor-Id`, `X-Actor-Name`, `X-Actor-Role` via `lib/actor.ts`.

**Rejection** is always permitted for any role.

### 7.3 Audit trail

`log_audit_event()` writes to `audit_log` (migration 0002): append-only via `audit_log_block_mutation` triggers. Actions include: `approve_claim`, `reject_claim`, `approve_denied`, `download_cms1500`, `download_statement`, `view_copilot`, `send_appeal_email`, `resolve_dispute`, `resubmit_corrected`.

Agent decisions also logged to `agent_runs` (including `human_review/decision` events).

### 7.4 Reviewer comments

Stored on `ClaimState` and `claims.reviewer_comment`, `reviewer_decision`, `reviewer_name`, `reviewer_role`.

## 8. Claims Lifecycle Workflows

### 8.1 Happy path

Upload → intake (high confidence) → eligibility (active) → coding → scrub (clean, risk < 60%) → submission (accepted) → reconciliation (variance OK) → fraud scan → `reconciled`. Patient statement if balance > 0.

### 8.2 Denial and appeal path

Submission denies with appealable CARC → `appealed`, GPT appeal letter → reviewer sends via `POST /send-appeal` (Resend) → payer replies to inbound address → webhook → AI `generate_dispute_reply` → optional escalation → `has_pending_dispute` → human resolves on `/disputes`.

### 8.3 Corrected claim / resubmission

Denied/appealed claim → `POST /claims/{id}/correct` with reason and optional line edits → new claim `frequency_code=7`, `original_payer_control_number=clearinghouse_ref`, pipeline restarts at **coded** → scrub → submission. Original linked via `corrected_by_claim_id`.

### 8.4 Dispute resolution path

1. Appeal email subject: `Appeal — Claim {8-char-id} — {patient}` (claim ID parsed on inbound).
2. Webhook acks immediately; `_process_dispute_inbound` fetches body, appends `payer_reply`, GPT `ai_reply`, persists to Storage + `dispute_threads`.
3. If payer affirms escalation after AI asked yes/no question → `has_pending_dispute=true`, static acknowledgment.
4. Supervisor resolves via `POST /disputes/{id}/resolve`.

## 9. Patient Management Module

**Auto-enrichment:** `upsert_patient_from_claim` after pipeline — match on `member_id` + `payer_name`, create encounter, link `claims.encounter_id`.

**Profiles (migration 0007):** Demographics, address, contacts, emergency contact, responsible party, insurance primary/secondary, notes. CRUD via `GET/PUT /patients/{id}`.

**Appointments:** `patient_appointments` — create/list/delete; upcoming sorted before past.

**Documents:** Metadata in `patient_documents`; files in Storage `patient-documents/{patient_id}/`; signed download URLs (1h expiry).

**A/R:** Patient balance on claim after reconciliation; statement PDF; optional Resend email to `ALERT_TO_EMAIL` (demo inbox).

## 10. The Claim Review Copilot

**Endpoint:** `POST /claims/{id}/chat`

**Context building:** `build_claim_context(state)` — markdown sections: identity, patient/payer, lines, extraction confidence, eligibility, coding, scrub findings, denial risk, adjudication, ERA, last 20 agent events. **No document bytes or storage paths sent to the model.**

**Models:** `MODEL_FAST` default; `MODEL_REASONING` when user message contains triggers (approve, reject, appeal, recommend, etc.).

**Output:** Structured `CopilotResponse`: `reply`, `citations`, `suggested_actions`.

**Cannot do:** Execute approve/reject/resubmit; invent facts not in context; access other claims; provide clinical treatment advice.

## 11. Frontend Architecture

| Route | Purpose | Data sources | Polling |
|---|---|---|---|
| `/` | Redirect to dashboard | — | — |
| `/dashboard` | Command center KPIs, recent claims | `listClaims`, `getAnalytics` | 10s |
| `/upload` | Single/batch superbill upload | `uploadClaim`, `uploadClaimBatch` | — |
| `/claims` | Searchable work list, CSV export | `searchClaims` | debounced |

| Route | Purpose | Data sources | Polling |
|---|---|---|---|
| `/claims/[id]` | Detail, pipeline, copilot, SSE feed | `getClaim`, SSE, `chatWithClaim`, `correctClaim`, `sendAppealEmail` | 5s + SSE |
| `/review` | HITL queue, bulk actions | `getReviewQueue`, `resumeClaim`, `bulkResume` | 15s |
| `/disputes` | Escalated email disputes | `getPendingDisputes`, `resolveDispute` | 15s |
| `/patients` | Directory | `listPatients` | debounced |
| `/patients/[id]` | Profile mini-EMR | `getPatient`, update, appointments, documents | — |
| `/analytics` | Billing intelligence charts | `getAnalytics` | 30s |
| `/ar` | Patient A/R aging | `getArAging` | 30s |

**SSE:** Only on claim detail — `EventSource` to `/claims/{id}/events`; handles `done` and `paused` terminal events; drives `PipelineDiagram`.

**State management:** React client state + polling; actor identity in `localStorage` (`lib/actor.ts`); Zod parsing in `lib/schemas.ts`.

**Stack note:** `package.json` specifies **Next.js 16.2.9** and **React 19** (README references Next.js 14 historically; code uses App Router throughout).

## 12. Database Schema

**Base tables (must exist in Supabase before seed/upload — not created by repo migrations):** `orgs`, `claims`, `patients`, `encounters`, `agent_runs`, `review_queue`.

**Migrations (additive, in order):**

| Migration | Purpose |
|---|---|
| 0001 | `claim_states` JSONB table (optional; app uses Storage instead) |
| 0002 | `audit_log` append-only with mutation triggers |
| 0003 | `claims` correction columns: `frequency_code`, `original_claim_id`, `correction_count`, `corrected_by_claim_id` |
| 0004 | A/R columns: `patient_responsibility`, `patient_balance`, `ar_status`, `statement_date` |
| 0005 | `appeal_email_sent` boolean |

| Migration | Purpose |
|---|---|
| 0006 | `dispute_threads` table + `claims.has_pending_dispute` |
| 0007 | Extended `patients` columns, `patient_documents`, `patient_appointments` |

**Storage bucket** `documents`**:** `claim-states/{uuid}.json`, `patient-documents/...`

**Key relationships:** `claims.encounter_id` → `encounters.patient_id` → `patients.id`; `agent_runs.claim_id`, `review_queue.claim_id`, `dispute_threads.claim_id` → `claims.id`.

## 13. Deployment Architecture

### 13.1 Local development

```
# Backend
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
# Configure backend/.env from .env.example
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
# Configure frontend/.env.local: NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
```

Train ML models (optional but recommended): `python -m app.ml.train` from `backend/`.

Seed demo data: `python data/synthetic/seed.py`. Generate superbills: `python data/synthetic/generate.py`.

### 13.2 Dual-environment dispute design

Resend inbound webhooks require a **public HTTPS URL**. Local `localhost:8000` cannot receive them. Recommended pattern:

1. Deploy backend (or a slim webhook forwarder) to **Render** with the same Supabase credentials.
2. Register Render URL + `/webhooks/resend-inbound` in Resend.
3. Render instance writes `dispute_thread` to Supabase Storage; local backend merges via `_apply_storage_dispute_fields()` on `GET /claims/{id}`.

Outbound emails (appeals, statements, dispute replies) work from local backend if `RESEND_API_KEY` and addresses are configured.

## 14. Technology Stack Summary

| Component | Version / source | Purpose |
|---|---|---|
| Python | 3.11 | Backend runtime |
| FastAPI | requirements.txt (unpinned) | REST API |
| Uvicorn | requirements.txt | ASGI server |
| sse-starlette | requirements.txt | SSE streaming |
| Pydantic | v2 | Schemas, ClaimState |
| LangGraph | requirements.txt | Agent orchestration |
| langchain-openai | requirements.txt | LangGraph OpenAI integration |
| OpenAI Python SDK | requirements.txt | Vision, structured outputs, chat |
| scikit-learn | requirements.txt | GradientBoosting, IsolationForest |
| SHAP | requirements.txt | Denial risk explanations |
| pandas / numpy | requirements.txt | Feature frames |
| ReportLab | 4.5.1 (installed) | CMS-1500, statement PDFs |
| pdf2image / Pillow | requirements.txt | PDF→image intake |
| Supabase Python | requirements.txt | Postgres + Storage client |
| Resend | requirements.txt | Transactional email |
| Faker | requirements.txt | Synthetic seed data |
| Next.js | 16.2.9 | Frontend framework |
| React | 19.2.7 | UI |
| TypeScript | ^5 | Type safety |
| Tailwind CSS | 3.4.1 | Styling |
| shadcn / radix-ui | 4.11.0 / 1.5.0 | UI primitives |
| Zod | 4.4.3 | API response validation |
| Framer Motion | 12.40.0 | Pipeline animation |
| Recharts | 3.8.1 | Analytics charts |
| react-dropzone | 15.0.0 | File upload |
| Supabase JS | ^2.108.1 | (dependency present; backend is primary data path) |

Model IDs: `MODEL_VISION`, `MODEL_REASONING`, `MODEL_FAST` — defaults in `.env.example`: `gpt-4o-mini`, `gpt-4o`, `gpt-4o-mini`.

## 15. Known Limitations and Future Work

- **Synthetic data only** — no real PHI; Faker-generated patients; demo NPI `1234567893`.
- **Single-org assumption** — first `orgs` row used for all claims.
- **Rule coverage** — curated CPT/ICD subset and NCCI pairs, not commercial scrubber completeness (~70k ICD codes).
- **No production authentication** — header-based demo RBAC only; missing headers default to manager (full approve).
- **ClaimState in Storage** — not Postgres `claim_states` table (migration exists but unused by current client).
- **Fraud history in-process** — resets on backend restart; not warehouse-persisted.
- **ML models** — must run `train.py` to create `.pkl` files; without them, fallback scores apply.
- **Free-tier constraints** — Supabase/Render/OpenAI rate limits affect demo throughput.
- **CORS** — backend allows only `http://localhost:3000`.

## 16. Compliance and Security Notes

- **Synthetic data only** — superbills, patients, and emails labeled as demo; no real PHI in logs by design.
- **HIPAA-aware choices** — audit log for PHI-adjacent downloads (CMS-1500, statement) and copilot access; SSN stored as last-4 only in patient schema; service role key server-side only.
- **Not production-certified** — no BAA, no encryption-at-rest guarantees beyond Supabase defaults, no penetration testing, no OCR/LLM data processing agreements for clinical use.
- **Secrets** — `backend/.env`, `frontend/.env.local` only; never committed.
- **Immutability** — audit_log triggers block UPDATE/DELETE; dispute and agent history append-oriented.

## Appendix A: API Endpoint Reference

| Method | Path | Description |
|---|---|---|
| GET | `/health` | Health check v0.2.0 |
| GET | `/auth/users` | Demo user roster |
| POST | `/claims/upload` | Single superbill upload |
| POST | `/claims/upload-batch` | Batch upload |
| GET | `/claims` | List recent claims (20) |
| GET | `/claims/search` | Paginated search + facets |
| GET | `/claims/export.csv` | Filtered CSV export |
| GET | `/claims/{id}` | Full ClaimState / row |
| GET | `/claims/{id}/events` | SSE agent events |

| Method | Path | Description |
|---|---|---|
| GET | `/claims/{id}/history` | Durable agent_runs trail |
| POST | `/claims/{id}/resume` | HITL approve/reject |
| POST | `/claims/{id}/correct` | File corrected claim |
| POST | `/claims/{id}/chat` | Review Copilot |
| POST | `/claims/{id}/send-appeal` | Email appeal letter |
| GET | `/claims/{id}/cms1500` | Download CMS-1500 PDF |
| GET | `/claims/{id}/statement` | Download patient statement |
| GET | `/review` | Open review queue |
| POST | `/review/bulk-resume` | Bulk approve/reject |
| GET | `/disputes/pending` | Escalated disputes |
| POST | `/disputes/{id}/resolve` | Clear dispute flag |
| POST | `/webhooks/resend-inbound` | Inbound email webhook |
| GET | `/analytics` | Billing analytics |
| GET | `/ar/aging` | Patient A/R aging |
| GET/PUT | `/patients`, `/patients/{id}` | Patient directory/detail |
| POST/DELETE | `/patients/{id}/appointments/...` | Appointments CRUD |
| GET/POST/DELETE | `/patients/{id}/documents/...` | Documents CRUD + upload |

## Appendix B: ClaimState Field Groups

See `backend/app/schemas/claim_state.py` — identity, extraction, eligibility, coding, scrub, denial risk, fraud, corrected-claim lineage, submission/adjudication, reconciliation/ERA, patient A/R, pipeline control (status, review, reviewer), dispute thread, `agent_events`, `errors`.

**Status enum:** `draft` → `extracted` → `coded` → `scrubbed` → `needs_review` → `submitted` → `denied` / `appealed` → `paid` / `reconciled`.

*Document generated from codebase review June 30, 2026. Every feature described above is implemented in the repository files cited.*